Is a Database Worth It?

nine questions, put to a text file, an indexed file,
PostgreSQL and openGauss, and timed by one program

Lin Yushen

12511937

SUSTech CS213 — Principles of Database Systems (H), Fall 2026

github.com/DevEason/SUSTech-CS213-DBMS-vs-File

Contents

Part I — Setting the stage

1 Introduction . . . 2
2 Method: one program, four layouts, two clocks . . . 2
2.1 The data, and the four ways of holding it . . . 2
2.2 Two clocks . . . 3
3 The nine questions . . . 3

Part II — The read comparison

4 Splitting text: the plain file loses six of seven . . . 4
5 A pre-built index: the file wins on speed . . . 4
6 Correctness: the file is fast, but wrong . . . 5
7 Database indexing: much faster, yet the file still wins . . . 6

Part III — Beyond the read comparison

8 The write: the file wins by doing no safety work . . . 7
9 PostgreSQL beats openGauss: a real win driven by a 13-year gap . . . 8
10 Scaling: the index's advantage grows with the data . . . 8

Part IV — Synthesis

11 Findings and conclusions . . . 9
11.1 What are the unique advantages of a DBMS? . . . 9
11.2 Which engine is better? . . . 10

Appendices

A Appendix A — why the client runs inside WSL . . . 11
B Appendix B — how to run the experiment . . . 11
C Appendix C — full measurement results . . . 13

1 Introduction

Since direct manipulation on file is more straightforward and flexible, why do we need a DBMS? This experiment tests the performance of the following 4 layouts of data: **csv-file**(plain text), **indexed-file**(build index on csv-file at start-up), **PostgreSQL** and **openGauss**.

In general, a **plain text file** re-parses every row for every query, resulting in poor speed. An **index built at start-up** makes the file the *fastest* of the four layouts at reading. For writing task, the file still *wins* (15× faster), since it does no safety work: no log, no version control, no constraints, no index. The file also answers *faster* for the word "fox", but with a *narrower* answer: it finds 1 film whereas the database finds 3.

However, when it comes to building an index for a query unknown in advance, file is unable and DBMS wins: **one line of SQL enables DBMS to do 106× less work**. As for the difference between two DBMS, it is the smallest lever: PostgreSQL leads openGauss by 1.5–2.1×, much less than the effect of index building.

Measurement results behind these claims are in Table 10, at Appendix C.

2 Method: one program, four layouts, two clocks

2.1 The data, and the four ways of holding it

One C++ program answers every question **four ways**: reading `*.csv` as text, reading it through indexes built at start-up, querying PostgreSQL, and querying openGauss. The data is FilmDB: ten tables, 76,131 rows, of which three tables are used for questions:

table	rows	its content	questions on it
`movies`	9,204	one film: id, title, country, year, runtime	every reading question
`people`	16,489	one person: id, name, born, died, gender	a joining question with `credits`, a writing question
`credits`	46,850	relationship per person per film, with how credited	a joining question with `people`
the other seven tables have 3,588 rows in total, untouched			

For the file layouts it opens a file; for the databases it opens a `libpq` connection. The only difference between two DBMS is the connection string (Figure 1).

Each question is asked several times after discarded warm-ups, with connection setup excluded, and the median is reported. Every operation also returns an answer, not only a time, and the four answers are compared.

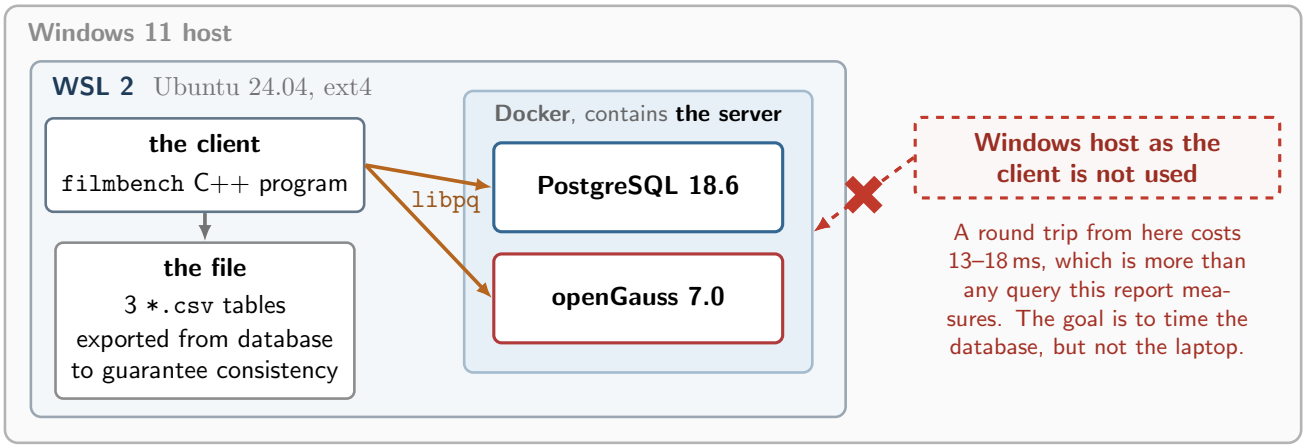


Figure 1: Where each part of the experiment runs. The red outline is the arrangement that was tried first and abandoned.

2.2 Two clocks

A **DBMS** measurement carries two numbers. C is what the caller waited, on one monotonic clock around the whole call; S is what the server spent, timed by the engine itself. They are related by

$$C = T + S + \varepsilon$$

where T is one empty round trip (a `SELECT 1`), measured once per session — 0.086 ms on PostgreSQL and 0.092 ms on openGauss — and ε is the residual, computed as $\varepsilon = C - T - S$, staying under 0.1 ms on every read. The engine reports S itself: with statement logging on it writes each statement's duration to its own log, and the run reads that log back and takes the median over the repetitions (Appendix B).

Remark: S cannot be recovered as $C - T$, because there is a gap ε between round trip T and execution time S — it includes the time for parsing and planning which occur before execution, but the server timer cannot contain such time spent (Figure 2).

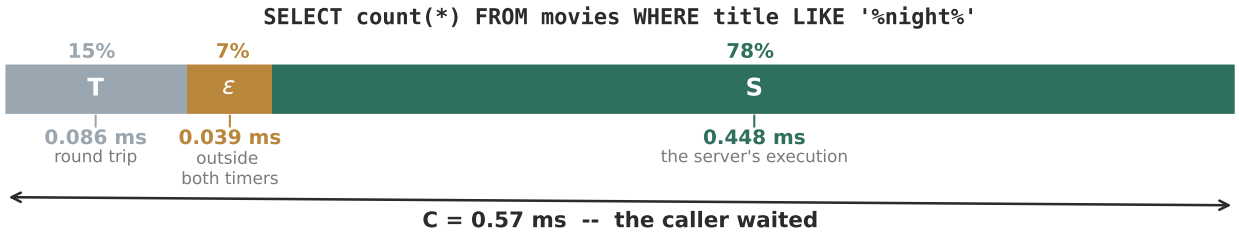


Figure 2: The caller's wait C for a scan on PostgreSQL, split into its three parts: the round trip T (15%), the residual ε (7%), and the server's execution S (78%). Parsing and planning are part of the residual gap ε.

The **file** layouts have no server and no wire — only the caller (`filmbench` C++ program) exists. So for them, only the caller time C exists and is reported. Every comparison between 4 layouts is therefore made on C. And S appears only where it is the subject, always labelled when used.

3 The nine questions

These nine questions separate three differences between a file and a database: whether rows are parsed once or on every question, whether an index can be built for a question nobody anticipated, and whether any safety work is done. Table 1 lists them in five families.

Table 1: Nine questions in five families.

#	the question asked	the database query	remark
scanning: no index can help			
T1	titles containing *night*	`title LIKE '%night%'`	a full scan
T2	the same, ignoring case	`title ILIKE '%night%'`	the same scan, more hits
an index answers it			
T3	titles starting with *The*	`title LIKE 'The%'`	a range scan
T4	the film called *The Dark Knight*	`title = 'The Dark Knight'`	a point probe
T5	the film with id 97	`movieid = 97`	a primary-key probe
a choice of access path			
T6.1	films about *fox*	`to_tsvector @@ to_tsquery`	no full-text index
T6.2	the same question	the same expression	with the index
T6.3	the same question	the same expression	index exists but forbidden
more than one table, or than one row			
T7	who worked on matching films	join to `credits`	a 3-table join
T8	how many films per country	`count(*) GROUP BY country`	a group-by
writing			
T9	rename every person	`UPDATE people SET ...`	a write

Comments on two design choices: T6 is the only question the four layouts can be made to disagree about, so it has three configurations (no index / index / index forbidden); T1 and T2 ask the same scan with different predicates (`LIKE` vs `ILIKE`), to show how much a database pays for a more expensive predicate compared with the file side.

4 Splitting text: the plain file loses six of seven

The plain CSV goes against both databases with nothing added to either side. The file side opens `movies.csv`, reads all 9,204 lines, splits each on its commas, and compares the title as text. The database stored those rows already parsed, in a fixed layout; the file must find the commas again on every query (that is why it loses). Table 2 gives the result.

Table 2: Seven questions, csv-file vs two DBMS. All values are *client time* in milliseconds.

		file/csv	PostgreSQL	openGauss	
T1	titles containing *night*	1.172	**0.573**	0.978	database, 2.0×
T2	the same, ignoring case	1.377	**1.222**	1.856	database, 1.1×
T3	titles starting with *The*	1.060	**0.270**	0.502	database, 3.9×
T4	the film called *The Dark Knight*	1.052	**0.137**	0.264	database, 7.7×
T7	who worked on matching films	5.995	**0.765**	1.358	database, 7.8×
T8	how many films per country	1.120	**0.728**	1.127	database, 1.5×
T5	the film with id 97	**0.0114**	0.124	0.258	**file, 11×**

PostgreSQL beats plain file in six of the questions, by **1.1** to **7.8** times.

T1 and T2 scan the same 9,204 rows. T2 asks the same question but ignores case, using `ILIKE` where T1 uses `LIKE`. That costs the database more: `ILIKE` folds every character to lower case before comparing, on all 9,204 rows, so PostgreSQL more than doubles (0.573 ms to 1.222 ms, +113%). The plain file barely moves (1.172 ms to 1.377 ms, +17%), because a case-sensitive and a case-insensitive compare cost almost the same in C++.

T5 is the only question where the plain file beats a database outright. The plain file reads and terminates when it finds `movieid = 97` — about 1% of the data — so waits 11 times less than PostgreSQL's 0.124 ms. In other words, the small `movieid` enables the file to scan much less and to find the target faster.

Two questions are not in this table: T6.1, where the file answers a narrower question (Section 6); T9 is the file doing no safety work (Section 8).

5 A pre-built index: the file wins on speed

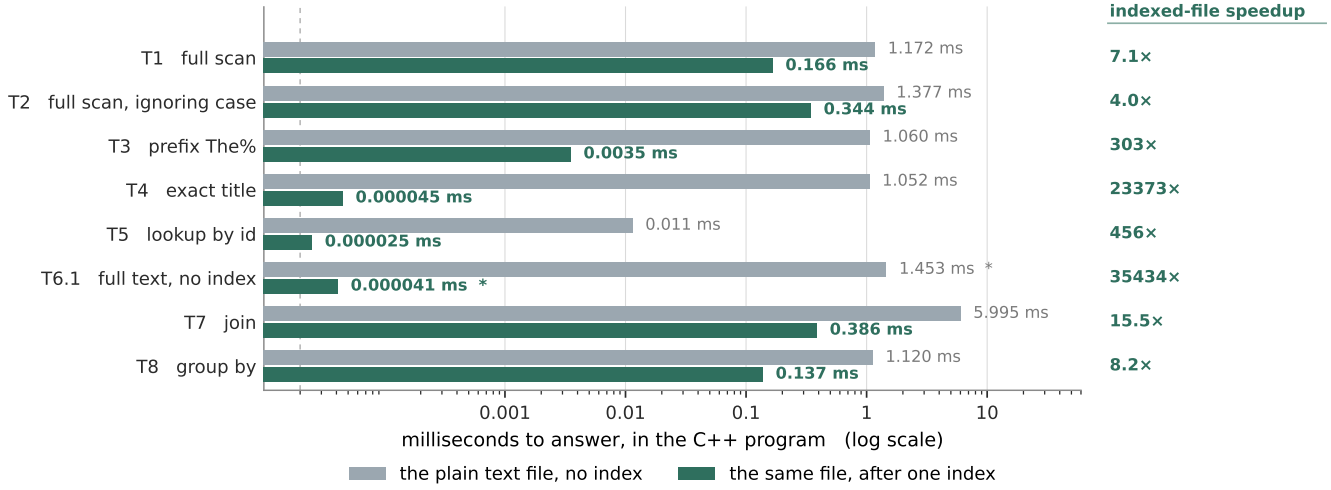
Parsing once at start-up makes the file the fastest layout. It reads `movies.csv` once, builds the three lookups in Table 3, and never opens the CSV again.

Table 3: The three lookups built at start-up, and what each serves.

lookup	serves
a hash from id to row	T5, the film with id 97
titles sorted, binary search	T3, titles starting with *The*; T4, the film called *The Dark Knight*
the same array, scanned for a substring	T2, the same, ignoring case

Figure 3 shows what each is worth. T3's **303** times is a binary search over 9,204 sorted titles; T4 is faster still, at **23373** times, because it lands on a single row.

T1 and T2 cannot use any index — a substring can start anywhere, so neither can binary-search the sorted array — and both still read all 9,204 records. They gain **7.1** and **4.0** times merely from reading records that were parsed once at start-up. T7 and T8 gain **15.5** and **8.2** times for the same reason: the indexed layout reads `people` and `credits` once at start-up, and counts a country from a fixed-width field.



* only finds 1 film where the databases find 3.
The indexed file's T4, T5 and T6.1 answers are 25-45 ns, most of which is the harness reaching the operation rather than the operation itself.

Figure 3: What the file gains from an index.

Table 4: The same file against each database, on the caller's clock, over the same **9,204** films.

		file/indexed ms	PostgreSQL ms (factor)	openGauss ms (factor)	the file's lead
T1	titles containing *night*	0.166	0.573 (**3.5**×)	0.978 (**5.9**×)	3.5–5.9×
T2	the same, ignoring case	0.344	1.222 (**3.6**×)	1.856 (**5.4**×)	3.6–5.4×
T3	titles starting with *The*	0.0035	0.270 (**77**×)	0.502 (**143**×)	77–143×
T4	the film called *The Dark Knight*	0.000045	0.137 (**3049**×)	0.264 (**5860**×)	3049–5860×
T5	the film with id 97	0.000025	0.124 (**4960**×)	0.258 (**10328**×)	**4960–10328**×
T7	who worked on matching films	0.386	0.765 (**2.0**×)	1.358 (**3.5**×)	2.0–3.5×
T8	how many films per country	0.137	0.728 (**5.3**×)	1.127 (**8.2**×)	5.3–8.2×

Each factor is the database's time divided by the file's; T4 and T5 use the times recorded at six decimals. The file's widest lead is T5, at **4960–10328**×.

The indexed file also beats both databases, on every question it can attempt. Table 4 compares the same 9,204 films on the caller's clock. T4 and T5 are mostly instrument. The indexed file answers them in 25–45 ns, and an empty call through the same path, with no operation inside (`--op noop`), costs about **18** ns. Most of T5's **25** ns is the harness reaching the operation, not the operation itself; a smaller share of T4's **45** ns. Appendix C gives the raw numbers, and Table 10 the whole matrix.

Table 4 omits T6.1 on purpose, and Figure 3 flags it with an asterisk. Section 6 reveals why.

6 Correctness: the file is fast, but wrong

The file's speed on T6.1 looks like a total victory. Asked for films about *fox*, the indexed file's word map finds **one**; the databases find **three**. The file got there in **0.000041** ms (Figure 3); a database that reads all 9,204 rows and stems each title takes **9.606** ms of caller time, and is right.

how it was answered	films found	what they were
file/indexed word map	1	*The Fox*
PostgreSQL / openGauss `to_tsvector`	3	plus *The Little Foxes*, *Prince of Foxes*
`title ILIKE '%fox%'`	7	plus *Foxfire*, *Foxy Brown*, *Foxcatcher*, *Firefox*

The database treats *Foxes* and *fox* as one word, because its full-text search runs each title through an English stemmer; the file's lookup compares whole words. The two are not disagreeing about a fact, they are answering different questions, and the file answers the narrower one.

The file is fast because it is doing less work, not because it found a better way to do the same work.

Closing that gap on the file side needs a stemming dictionary and a stop-word list — a search engine, not a benchmark. This is where **declarative** and **imperative** diverge: the SQL query states *what* is wanted, and the search engine figures out *how to get*; the C++ code must spell out *how to get*, and still gets it wrong. Forty lines of C++ reproduce the tokeniser but not the stemmer; one predicate, `to_tsvector('english', title) @@ to_tsquery('english', 'fox')`, gets both. The database's answer is right and slow; the next section asks whether an index can make it fast.

7 Database indexing: much faster, yet the file still wins

T6 is the only question asked three ways: T6.1 with no full-text index, T6.2 after one `CREATE INDEX`, and T6.3 with the index forbidden to the planner.

The shipped schema has nothing to help a text search. **T6.1** is a full scan: 9.444 ms of server time on PostgreSQL, 14.265 ms on openGauss. One line creates a `GIN` index over the text:

```
create index ix_ft on movies using gin (to_tsvector('english', title));
```

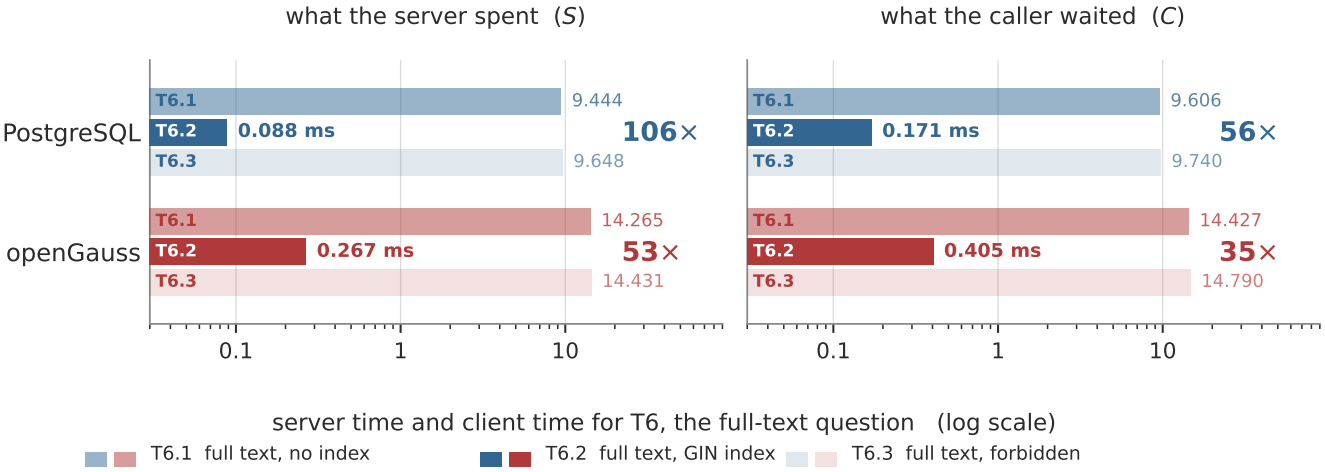


Figure 4: One index on one question, on the server's clock and the caller's.

On the server, the index is a miracle. Figure 4 puts both clocks side by side: execution time falls from 9.444 ms to 0.089 ms — **106** times less work.

T6.3 proves it was the index. The planner is forced to ignore the index; server time returns to 9.648 ms against T6.1's 9.444 ms. Two independent routes to "no index", agreeing within 2%.

But on the caller's clock the miracle shrinks. The wait falls from 9.606 ms to 0.171 ms — only **56** times less waiting. The missing 50 times are lost to the network round trip.

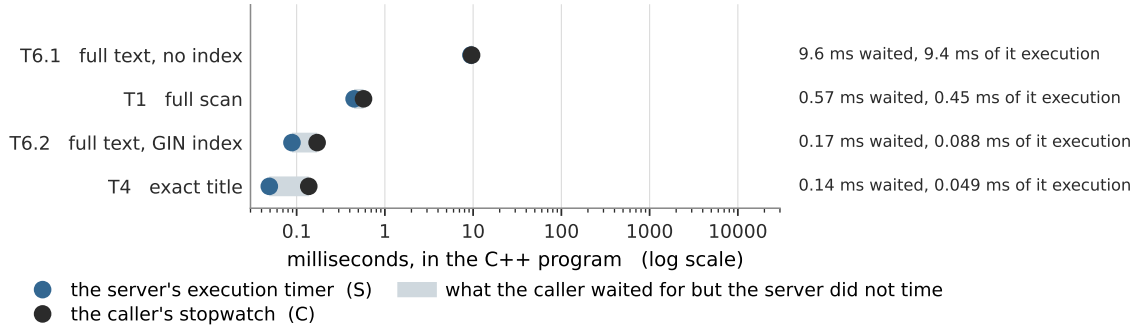


Figure 5: The two clocks on four read questions. As queries get faster, server execution (S) falls from **98**% to **36**% of the wait, leaving the fixed round trip to dominate the caller's wait (C).

The faster the index, the more the trip is the bottleneck. Figure 5 shows the limit. On the slowest scan (T6.1) the server is **98**% of the wait. On the fastest lookup (T4), the server shrinks to **36**%, and the fixed round trip takes over **62**%.

This network floor explains why the file still wins. Even at its physical limit of 0.171 ms, the indexed database is still more than **4,000** times slower than the indexed file (0.000041 ms, on one film rather than three), because the file pays no network overhead.

The on-demand index is a real self-rescue, but it cannot beat a local file that has no wire. It buys a different power: answering a question the file was never built for — and, as Section 6 shows, answering it correctly.

The read comparison ends here. The three sections below ask what a read comparison cannot: which layout wins when the question *writes* (Section 8), which *engine* is faster (Section 9), and whether the conclusions survive when the data grows **150** times (Section 10).

8 The write: the file wins by doing no safety work

T9 changes every "T" in every person's name to "Ttt": 16,489 rows and 1,321 characters. The file side is a loop over a text file; the database side is one `UPDATE` with no `WHERE` clause.

T9, rename every person	**what the caller waited**	what the server spent	the wire
file/csv rewriting the file	**3.656 ms**	—	—
PostgreSQL `UPDATE`	56.448 ms	55.279 ms	0.086 ms
openGauss `UPDATE`	99.132 ms	97.860 ms	0.092 ms

The file wins by 15 times against PostgreSQL and 26 against openGauss, for five reasons:

- there is no index on the names, so there is nothing to update;
- there is no write-ahead log, so there is nothing to flush before answering;
- no old row versions have to be kept;
- there are no constraints to check, because the file has none;
- one sequential pass over one file, against 16,489 scattered row updates.

PostgreSQL pays roughly 3.2 μs per row for this safety work. The file side pays nothing; it simply writes a fresh copy and renames it into place — acting as a lightweight, file-side `ROLLBACK` (Appendix B). Two caveats, recorded in Appendix C: each repetition runs inside `BEGIN` and `ROLLBACK`, part of PostgreSQL's 1.08 ms of unaccounted waiting; and every database run carries statement logging, which the file side does not pay for.

9 PostgreSQL beats openGauss: a real win driven by a 13-year gap

With both engines configured identically, PostgreSQL leads on all nine questions by 1.5 to 2.1 times (Table 5). Out of the box, openGauss arrived with `shared_buffers` at 1024 MB against PostgreSQL's 128 MB. To avoid measuring configuration files, both were set to `shared_buffers` 128 MB, `work_mem` 64 MB, and `effective_cache_size` 4 GB, then restarted.

Table 5: PostgreSQL against openGauss on the same clock. Ratio = openGauss / PostgreSQL.

#	the question	PostgreSQL	openGauss	ratio
T1	titles containing *night*	0.573 ms	0.978 ms	1.7×
T2	the same, ignoring case	1.222 ms	1.856 ms	1.5×
T3	titles starting with *The*	0.270 ms	0.502 ms	1.9×
T4	the film called *The Dark Knight*	0.137 ms	0.264 ms	1.9×
T5	the film with id 97	0.124 ms	0.258 ms	2.1×
T6, films about fox*, in its three configurations*				
T6.1	films about *fox*, no full-text index	9.606 ms	14.427 ms	1.5×
T6.2	the same question, with the index	0.171 ms	0.405 ms	**2.4**×
T6.3	the same question, index exists but forbidden	9.740 ms	14.790 ms	1.5×
T7	who worked on matching films	0.765 ms	1.358 ms	1.8×
T8	how many films per country	0.728 ms	1.127 ms	1.5×
T9	rename every person	56.448 ms	99.132 ms	1.8×

A ranking this close is easy to over-read. **openGauss** 7.0 reports a `server_version` of 9.2: the PostgreSQL kernel it forked from, not a release date. Against 18.6, the gap is not a version number. It is **13** years of storage-engine work.

The plans on T3 show where. Both engines hold the same B-tree on the title and return the same **1,449** films, but they answer it in different ways. **PostgreSQL** runs an *Index Only Scan* and completely avoids table fetches; **openGauss** runs a *Bitmap Index Scan*, then reads almost the entire table (Table 6).

Table 6: One B-tree, two plans. **PostgreSQL** avoids the table entirely; **openGauss** reads **71** of its **72** pages.

question	engine	rows returned	index pages	heap blocks	table pages
T3	PostgreSQL	1,449	14	**0**	72
T3	openGauss	1,449	13	**71**	72

Both engines answer seven of the nine questions in under two milliseconds: on this data, use whichever is already running. The engine is the smaller lever. The widest engine gap is **2.1**× of caller time; one `CREATE INDEX` was worth **106**× of server time — and **56**× of the caller's. What happens when the data grows 150 times? (Section 10)

10 Scaling: the index's advantage grows with the data

Section 9 showed that choosing an engine yields a mere **2**× difference, against **106**× for the index. **But what happens to that 106× advantage when the data grows 150 times?** Now the films table is copied **150 times** into `movies_big` with fresh ids: **1,380,600 rows**, 123 MB with the standard indexes, and 137 MB once the `GIN` index is added. That crosses the 128 MB buffer pool.

Only three questions can test this, because a question scales only if its answer grows with the table (a primary-key lookup always returns one row, so it cannot scale). Table 7 gives the results:

Table 7: The three questions that scale, measured at both sizes. The second time column is the engine's `EXPLAIN (ANALYZE)` execution time — the channel that also reports the plan and its buffers, which is what a growth comparison needs.

#	question	rows	size	caller wait, ms	`EXPLAIN`, ms	vs. 9,204 rows
the 9,204-row table, for reference						
T5	lookup by id	9,204	1.5 MB	0.151	0.012	—
T3	prefix `The%`	9,204	1.5 MB	0.238	0.133	—
T6.1	full text, no index	9,204	1.5 MB	9.582	9.265	—
`movies_big`, 150 times the rows						
T5	lookup by id	1,380,600	123 MB	0.128	0.009	did not move
T3	prefix `The%`	1,380,600	123 MB	34.194	41.016	**310**× slower
T6.1	full text, no index	1,380,600	123 MB	1,393.106	1,398.492	**151**× slower
T6.2	full text, `GIN` index	1,380,600	137 MB	0.385	0.169	**8,300**× faster than its scan

The primary-key probe did not move. For the queries that must scan:

- **Prefix search is 310× slower.** More than proportional to the **150**× rows, because its result set exploded from **1,449** to **217,350**.
- **Unindexed full-text scan is 151× slower,** scaling exactly with the row count.
- **The `GIN` index is 8,300× faster** than that scan, up from **106**× on the small table.

An index's advantage grows with the data. The lever that was already the largest in Section 7 becomes the only one that matters at scale.

(Two limitations: The table exceeds `shared_buffers`, but every plan reported `shared read=0` because the OS page cache absorbed the difference. And this check covers databases only, since no enlarged file copy exists for the file layouts.)

11 Findings and conclusions

Table 8: Who wins each question, on caller's clock and correctness.

#	the question	fastest	answer correct?	how far ahead of **PostgreSQL**
T1–T5, T7–T8	titles, ids, joins, aggregates	**file/indexed**	Yes	**2.0**× to **4960**×
T6.1	films about *fox*	**file/indexed**	**No** (**1** vs **3**)	**234,283**×, but **wrong**
T9	rename every person	**file/csv**	Yes	**15**×

The indexed file wins on speed, but not always on truth. Table 8 is the scoreboard. On T6.1 its extreme speed comes at a fatal cost: it returns the wrong answer. The database is the only layout that survives both columns. Here are the direct answers to the assignment's two questions.

11.1 What are the unique advantages of a DBMS?

Not raw speed: on data that fits in memory, an indexed CSV was the fastest way to read. A DBMS wins on four structural capabilities the file cannot match:

- **Indexing on demand.** One line of SQL cut server work **106**× for a query the index was never built for (Section 7).
- **Correct semantics.** The database stems *fox* and *Foxes* as one word; the file's string match fails (Section 6).
- **Transactional update.** Atomic, durable writes with rollback, against a file copy-and-rename (Section 8).
- **Constraints. 63** of them, none of which a CSV can express or enforce (Section 8).

11.2 Which engine is better?

PostgreSQL. It leads openGauss on all nine questions by **1.5** to **2.1** times under identical settings, reflecting a thirteen-year architectural gap (Section 9). **Scope:** this verdict holds for one client, one query at a time, on memory-resident data. For this data, use whichever is already running.

So is a database worth it? On nine questions over 11 MB of memory-resident data, it is mostly a slower way to read a file. It earns its cost only when:

- a query needs an index that could not have been anticipated,
- the answer must be correct, not merely fast, or
- an operation must be atomic, durable and rollback-capable, or enforce constraints.

Although the nine questions were chosen to be easy — which flatters the file — all three conditions still occurred.

The boundary. At the small end, T4 and T5 hit the instrument's floor (tens of nanoseconds). At the large end, Section 10 confirms that the index's advantage grows with data. Across two runs all **41** answers were identical and the index effect re-measured at **120** against **106**, though every absolute time moved by a median of **1.52** (Appendix C).

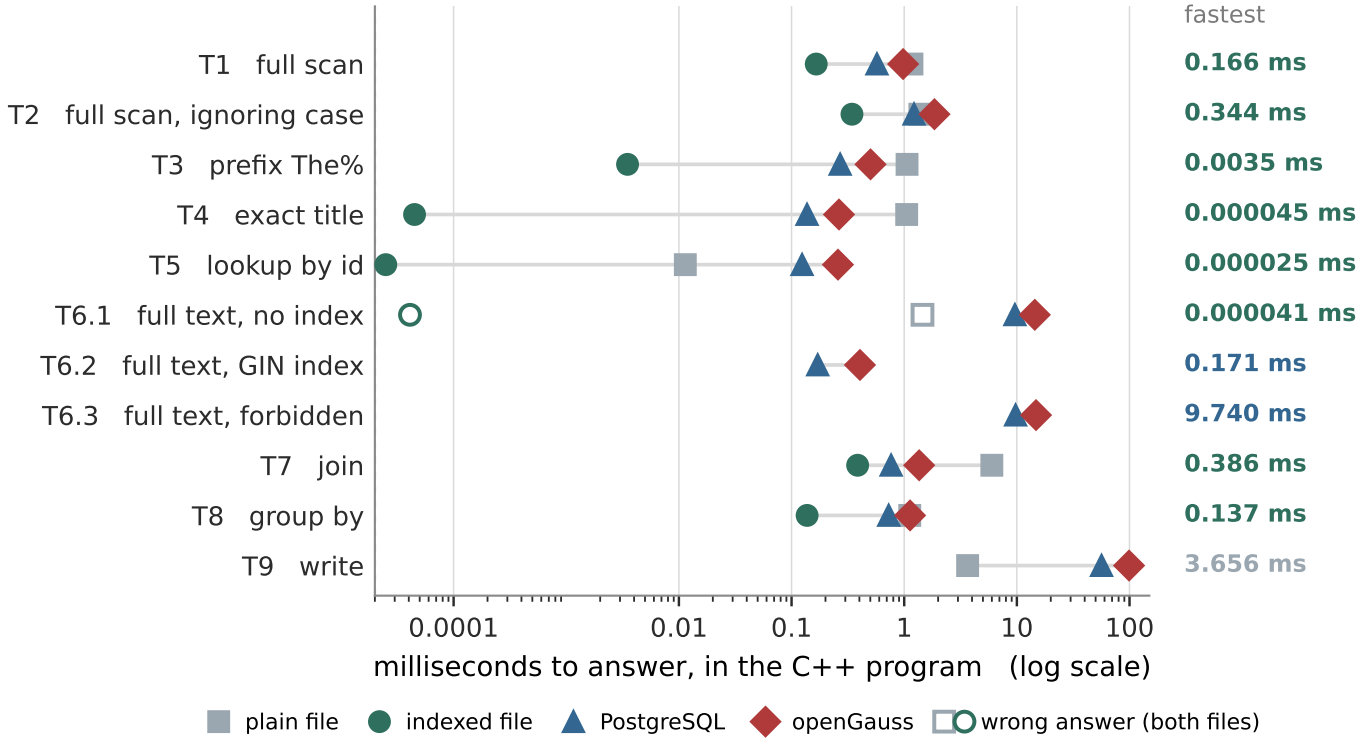


Figure 6: Every question, every system, on the one clock they share. Missing markers indicate that the layout cannot attempt the query.

Figure 6 draws that table question by question; its eleven rows carry **39** of the **44** measurements in Table 10.

Table 9 maps each requirement of the project to the section that answers it.

Table 9: Where each requirement of the assignment is answered.

Project requirement	Where addressed
Data in a database *and* in a file	Section 2
Retrieval comparison: SQL vs a C/C++ program	Sections 4–7
Update comparison: `UPDATE` vs the file	Section 8
Other comparisons (reorganised storage, joins, aggregates, full text)	Sections 5 and 6
PostgreSQL vs openGauss	Section 9
Unique advantages of a DBMS	Section 11

A Appendix A — why the client runs inside WSL

Both databases run in Docker containers inside a WSL 2 virtual machine. A Windows client talking to `127.0.0.1:5434` read **14** ms per query whatever the query asked, because one empty `SELECT 1` across that boundary costs **16** ms against **0.10** ms from inside WSL; reading the data files across the Windows mount was about three times as expensive as reading them from ext4.

The client runs inside the virtual machine, beside the containers and outside Docker. **Every number in this report comes from that client.**

B Appendix B — how to run the experiment

Environment: WSL 2, g++ 13.3, libpq 16. The harness (`filmbench.cpp`) is the only program this report ships; the dataset and its loader come with the course. Everything this appendix needs is also at `github.com/DevEason/SUSTech-CS213-DBMS-vs-File`.

1. Build the harness. It must exist before `export` runs.

```
g++ -std=c++17 -O2 -I/usr/include/postgresql filmbench.cpp -o filmbench -lpq
```

2. Load both engines, then export. `filmdb-load.sql` is the course's loader; it drops and reloads the ten tables, so both engines start from identical bytes. The file layouts own no data: they read CSVs written by `export`, which is what makes the four systems comparable.

```
psql -f filmdb-load.sql     # once per engine; drops and reloads the ten tables
mkdir -p /root/filmdata
./filmbench export --dsn $DSN --data /root/filmdata     # the CSVs the file side reads
```

Check 1 — the same rows on both engines. Each table is reduced to one MD5 over every column of every row, ordered by primary key; point `$DSN` at each engine in turn and compare the lines they print.

```
psql "$DSN" -At -F'|' -c "select 'movies', count(*),
md5(string_agg(movieid||':'||title||':'||country||':'||
year_released||':'||coalesce(runtime::text,'~'),';;' order by movieid)) from movies"
```

One statement of this shape per table: the ten tables print ten lines, and both engines must print the same ten. The digests that matched are listed in Appendix C. A mismatch stops the run here, before anything is timed.

3. Align, restart and measure. `shared_buffers` is the one setting that lives on the server, so it is set once per engine and a restart is what applies it. The other three are session settings and the harness issues them itself, sending both engines' names and letting each ignore the ones it does not have.

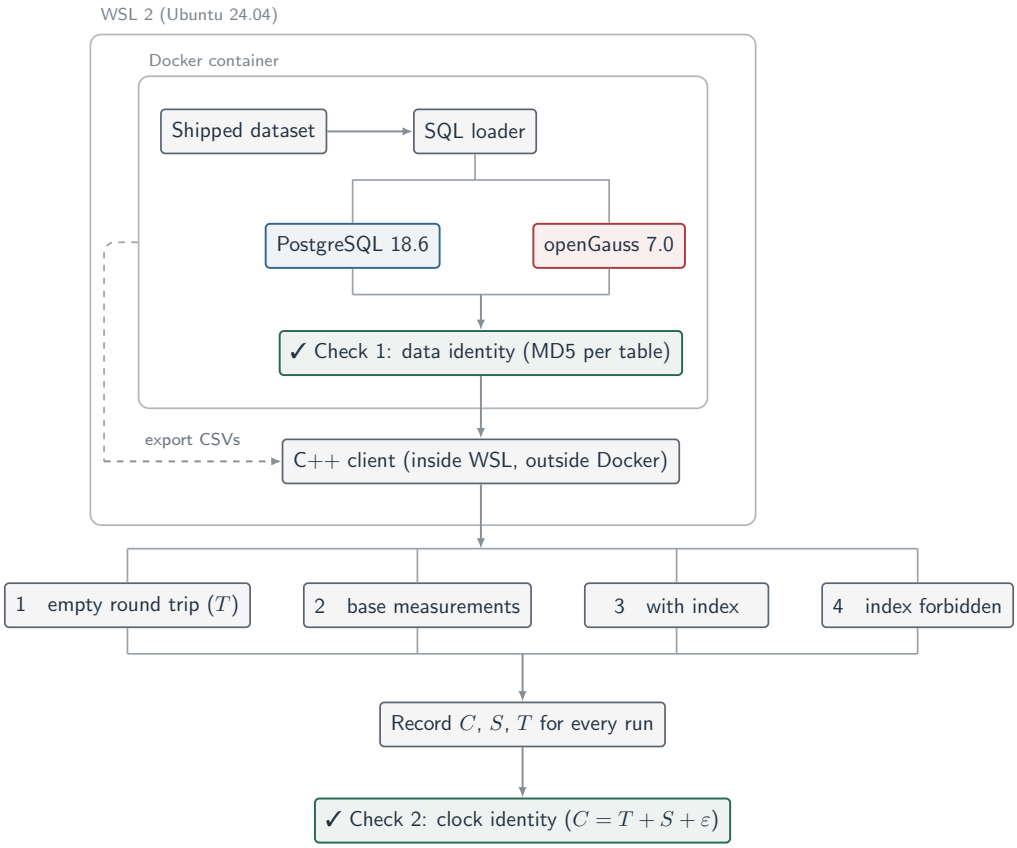


Figure 7: The experiment workflow. Both engines run in Docker inside WSL, the client in WSL outside it. The file layouts read CSVs exported from the database, so all four systems answer from identical bytes. Two automated checks stop the pipeline if a data or timing assumption breaks.

```
ALTER SYSTEM SET shared_buffers = '128MB';     # both engines accept it
# restart both engines, then one empty round trip:  this number is T
./filmbench ping --dsn $DSN --runs 20     # T, one number per engine
./filmbench file --data /root/filmdata --layout csv|indexed --op OP --out file-rows.csv
```

4. Measure, with `S` read from the engine's own log. One invocation does the whole row: it times the query, switches the session's statement log on, prints the mark that separates this run from earlier ones, runs the log command, and writes a tidy row of `C`, `S`, `T` and the answer.

```
./filmbench db --dsn $DSN --op OP --term T --runs 20 --warmup 3 \
--label postgresql|opengauss --transfer $T --match '%night%' \
--log-server-timing --log-cmd 'docker logs postgresql 2>&1' --out rows.csv
# --op takes one of:  scan scani prefix exact lookup ftsearch join groupby update
# --match is whatever tells the statement apart:  '%night%', 'movieid = 97',
# 'to_tsquery', 'GROUP BY country'.  --key 97 for T5, --no-index for T6.3.
```

T6.2 and T6.3 are the same command with one index added and one flag changed: the index has to exist for T6.2 and be forbidden for T6.3.

```
psql $DSN -c "CREATE INDEX ix_movies_ft \
ON movies USING gin (to_tsvector('english', title))"
psql $DSN -c "ANALYZE movies;"     # statistics for that plan
# then --no-index for T6.3, and DROP INDEX when the run is over
```

For **openGauss** pass instead

```
--log-cmd "docker exec opengauss bash -lc 'cat /var/lib/opengauss/data/pg_log/*.log'"
```

The harness keeps the durations after the last mark, drops its own `SET` and `EXPLAIN` lines, and takes the median with the same even-count rule it uses for `C` — so the two figures describe the same execution. Without `--log-cmd` it reports `EXPLAIN (ANALYZE)`'s execution time in column `explain_ms` instead, which is the channel Section 10 uses, not this one. Figure 7 draws the flow and the log as one picture.

Check 2 — the two clocks agree. With `C`, `S` and `T` in one row, the identity can be checked by the program that wrote them:

```
./filmbench verify rows.csv
```

It prints the rows whose residual falls outside a quarter of a millisecond or 5% of their own duration, whichever is larger, and exits non-zero if any does. On the **16** read queries of Appendix C it prints nothing; the two writes are exempt, because their residual is the `BEGIN/ROLLBACK` commit `S` does not cover (Section 8). A flagged row is a row whose two clocks are not measuring the same interval, and it is not reported.

File-side ROLLBACK. For T9 the file side writes a fresh copy and renames it into place, leaving the original intact, so every repetition starts from the same bytes.

The nine questions. One `--op` value per question, and its parameters: the SQL each expands to is `sqlFor()` in `filmbench.cpp`, and Section 3 says what the question means.

```
# T1, T2:  substring scans
./filmbench file --data DIR --layout indexed --op scan --term night
./filmbench db --dsn $DSN --op scani --term night
# T3, T4:  prefix and exact title
./filmbench db --dsn $DSN --op prefix --term The
./filmbench db --dsn $DSN --op exact --term "The Dark Knight"
# T5:  primary-key lookup
./filmbench file --data DIR --layout indexed --op lookup --key 97
./filmbench db --dsn $DSN --op lookup --key 97
# T6.1, T6.2, T6.3:  full text, as is, then after the index of step 4, then forbidden
./filmbench db --dsn $DSN --op ftsearch --term fox
./filmbench db --dsn $DSN --op ftsearch --term fox --no-index
# T7, T8:  join and group-by
./filmbench db --dsn $DSN --op join --term night
./filmbench db --dsn $DSN --op groupby
# T9:  update
./filmbench file --data DIR --layout csv --op update
./filmbench db --dsn $DSN --op update
```

C Appendix C — full measurement results

Machine: Intel Core Ultra 9 275HX (24 cores, 31.4 GB RAM), Windows 11 host, Ubuntu 24.04 under WSL 2, g++ 13.3, libpq 16; `postgresql` 18.6 on port 5434 and `opengauss` 7.0.0-RC3 on port 5432, both in Docker, one bridge network. The client ran inside WSL, outside Docker (Appendix A).

Data provenance. Every comparison between the four systems uses the caller's wait C (the harness's own monotonic clock). The decomposition of wait time into server work and residual (Table 11, Figure 2), and the index effect quoted in Sections 7 and 11, read S from the engine's statement log. Mechanism and scaling analyses (Table 6 plans, Table 7 execution times, Section 10 ratios) use `EXPLAIN (ANALYZE)`. No single comparison mixes the two channels.

Data consistency. Ten tables matched identically: an MD5 over every column of every row, ordered by primary key, text compared under collation `C`. 76,131 rows; both engines report 37 columns, 63 constraints, 21 indexes, encoding UTF8 and collation `C/C`.

table	rows	MD5 of every column of every row
`movies`	9,204	`1f936f4ea7300e2292da96757d63e6cf`
`people`	16,489	`ecb033910096143e21d38fd248d9399c`
`credits`	46,850	`9bcecad02ad88d8735b18a9ff738aaef`
the other seven, 3,588 rows		`63c18643af75d34fd1d18e5ce4377db5`
all ten, identical on both engines		`95ecffd81b82b56a412b64cf102028c3`

Second session. The whole experiment was run a second time, in its own session: all **41** answers matched between the two, the T6 disagreement included, and so did all **24** environment checks. Of the **37** client times the two sessions share, the median moved **1.52**× (1.03 to 2.24) and the server clock **1.56**×; repeats agree to ±4%. **Every database ratio survived:** the index effect re-measured at **120** against **106**, and **53** against **54** on openGauss. Two file-side ratios did not, their numerator sitting in microseconds. Absolute times belong to their session; ratios transfer across them.

The full matrix. Every cell as `filmbench.cpp` recorded it. A `---` is a measurement that does not exist, not one that was skipped: the indexed file cannot answer T9, having stopped reading the file T9 rewrites, and the file layouts have no T6.2 or T6.3, that index being the database's.

Table 10: Every measurement in the experiment, on the caller's clock.

#	question	answer	client time C, ms			
			openGauss	PostgreSQL	file/csv	file/indexed
T1	titles containing *night*	21	0.9781	0.5727	1.1719	0.1656
T2	the same, ignoring case	103	1.8557	1.2222	1.3766	0.3437
T3	titles starting with *The*	1449	0.5023	0.2700	1.0597	0.0035
T4	the film called *The Dark Knight*	1	0.2637	0.1372	1.0518	0.000045
T5	the film with id 97	1	0.2582	0.1240	0.0114	0.000025
T6.1	films about *fox*, no full-text index	3[†]	14.4266	9.6056	1.4528	0.000041
T6.2	the same, with the index	3	0.4048	0.1706	—	—
T6.3	the same, index forbidden	3	14.7905	9.7396	—	—
T7	who worked on matching films	128	1.3579	0.7651	5.9945	0.3860
T8	how many films per country	85	1.1270	0.7281	1.1199	0.1374
T9	rename every person	16489	99.1316	56.4479	3.6556	—

The three rows in tens of nanoseconds are given to six decimals of a millisecond — a nanosecond each, below the harness's own call path.

[†] the layouts disagree here: both files match whole words and find **1** film, the databases match word stems and find **3**; the answer column shows the databases' answer (Section 6).

Time decomposition. Table 11 splits each database time into the caller's wait `C`, the server's own time `S` read back from the engine's statement log, and the empty round trip `T`.

Table 11: The same two clocks, split by what each part costs.

		PostgreSQL			openGauss		
question		C	S	T	C	S	T
T1	full scan	0.5727	0.4480	0.0855	0.9781	0.8080	0.0920
T2	full scan, ignoring case	1.2222	1.0420	0.0855	1.8557	1.6870	0.0920
T3	prefix `The%`	0.2700	0.1230	0.0855	0.5023	0.3495	0.0920
T4	exact title	0.1372	0.0490	0.0855	0.2637	0.1395	0.0920
T5	lookup by id	0.1240	0.0385	0.0855	0.2582	0.1350	0.0920
T6.1	full text, no index	9.6056	9.4440	0.0855	14.4266	14.2650	0.0920
T6.2	full text, `GIN` index	0.1706	0.0885	0.0855	0.4048	0.2670	0.0920
T6.3	full text, forbidden	9.7396	9.6475	0.0855	14.7905	14.4315	0.0920
T7	join	0.7651	0.6270	0.0855	1.3579	1.1980	0.0920
T8	group by	0.7281	0.5705	0.0855	1.1270	0.9575	0.0920
T9	write	56.4479	55.2790	0.0855	99.1316	97.8600	0.0920

Residual and parsing.

- **Residual ε.** Under **0.1** ms on all **16** reads; the two writes reach **+1.08** ms on **PostgreSQL** and **+1.18** ms on **openGauss**, the commit outside `S`.
- **Parsing.** A **44** kB `IN` list costs 0.673 ms of execution, a comment of the same size 0.058 ms: planning scales with expressions, not bytes — which is why `S` is measured by the engine and never inferred as `C` − `T`.

Limitation. The queried tables hold 3.6 MB of rows (3,653,632 bytes) and 11 MB with their indexes (10,936,320 bytes), inside the 128 MB pool, so no read reached disk; Section 10 crossed the pool without provoking one either. Every number here is a memory-speed comparison.